

Title: Message Passing and Receiving Using Modulator (part 1: Transmitter Side)

Objectives:

1. To understand the concept of message encoding and decoding.
2. To understand the concept of serial transmission and reception of message.
3. To develop understanding of data transmission and reception process.

Theory:

On the transmitter side, the main objective is to convert the original message into a form suitable for efficient transmission over a communication channel. The process begins with the message signal (baseband signal), which may be an audio, video, or data signal. This signal typically has low frequency and cannot travel long distances effectively, so it needs to be modified before transmission.

First, the message signal is passed through a signal conditioning stage, where it may be amplified or filtered to remove unwanted noise and improve quality. Then, a carrier signal—a high-frequency sinusoidal wave generated by an oscillator is produced. This carrier plays a crucial role in carrying the message over long distances.

The modulator then combines the message signal with the carrier signal. Depending on the modulation technique, it varies a specific parameter of the carrier:

- In Amplitude Modulation (AM), the amplitude of the carrier changes.
- In Frequency Modulation (FM), the frequency varies.
- In Phase Modulation (PM), the phase is altered.

As a result, a modulated signal is generated, which contains the information of the original message but in a higher frequency form suitable for transmission. This modulated signal is then passed through a power amplifier to increase its strength. Finally, the amplified signal is transmitted through an antenna (in wireless systems) or a transmission medium like a cable. This completes the transmitter-side process, ensuring that the message is properly prepared and efficiently sent toward the receiver.

Block Diagram of a Transmitter

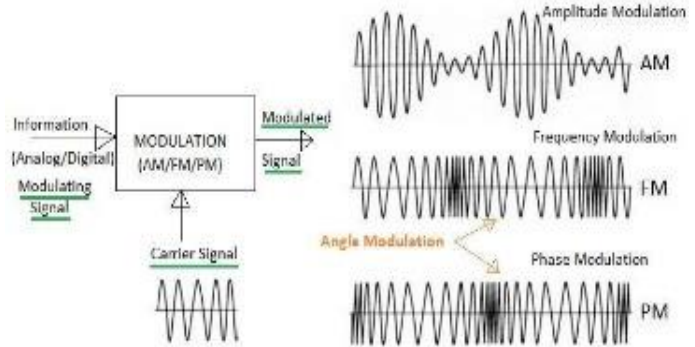
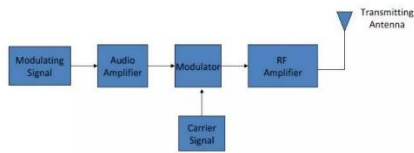


Fig: Message Passing and Receiving Using Modulator as Transmitter Side

Code:

```

function dn = asc2bn(txt)
dec = double(txt);
p2 = 2.^(0:-1:-7);
B = mod(floor(p2' * dec), 2);
dn = reshape(B,1,numel(B));
end

clc;
clear;
close all;

% message
Transmitted_Message = '23-54972-3';

% user choice
choice = input('Enter 1 for Synchronous, 2 for Asynchronous: ');

% bit duration 1 sec
Tb = 1;           % 1 second per bit
samples_per_bit = 100; % for smooth plotting

if choice == 1
    x = asc2bn(Transmitted_Message);

    disp('Synchronous Binary Data:');
    disp(x);

    signal_type = 'Synchronous Digital Signal';
elseif choice == 2
    n_chars = length(Transmitted_Message);
    x = zeros(1, n_chars * 10);

    for i = 1:n_chars
        char_bin = asc2bn(Transmitted_Message(i)); % 8-bit

        frame = [0 char_bin 1]; % start + data + stop

        idx = (i-1)*10 + (1:10);
        x(idx) = frame;
    end

    disp('Asynchronous 10-bit Data:');
    disp(x);

    signal_type = 'Asynchronous Digital Signal (10-bit frames)';
else
    error('Invalid choice');
end
  
```

```

|
bit = zeros(1, length(x) * samples_per_bit);

for n = 1:length(x)
    idx = (n-1)*samples_per_bit + (1:samples_per_bit);

    if x(n) == 1
        bit(idx) = 1; % logical high
    else
        bit(idx) = 0; % logical low
    end
end

t = linspace(0, Tb*length(x), length(bit));

figure;

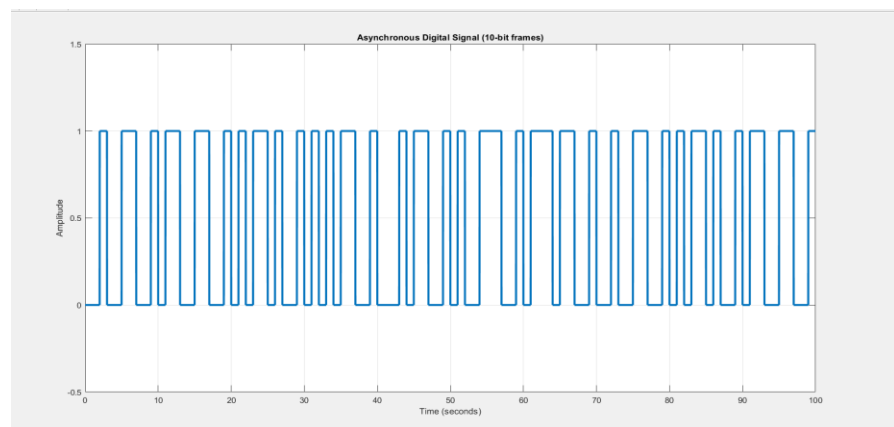
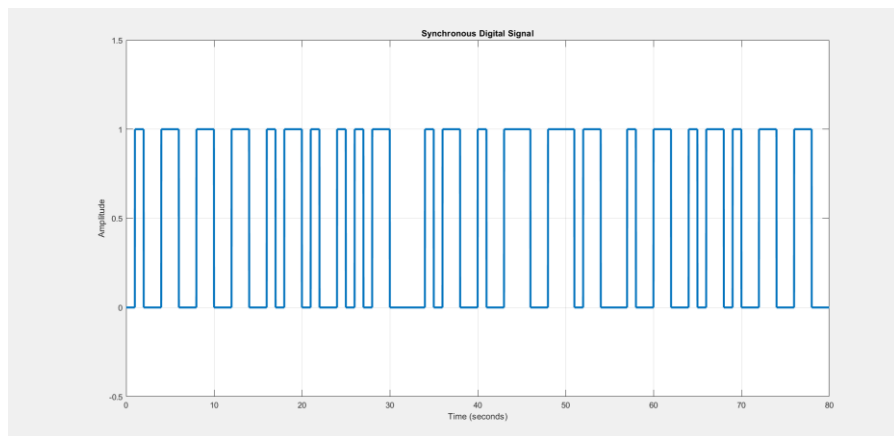
plot(t, bit, 'LineWidth', 2.5);
grid on;
axis([0 Tb*length(x) -0.5 1.5]);

xlabel('Time (seconds)');
ylabel('Amplitude');
title(signal_type);

disp('Figure is open. Press ENTER in Command Window to close..');
input('','s');

```

Simulation Figure:



Result analysis/discussion:

The simulation successfully demonstrated the generation of digital signals for a text-based input (23-54972-3) using two distinct transmission methods. The text input was converted into its ASCII-based binary equivalent using the `asc2bn` function. When Option 1 was selected, the system produced a continuous stream of bits. The resulting plot showed a Unipolar Non-Return-to-Zero (NRZ) signal where each bit occupied exactly 1 second of duration. When Option 2 was selected, the 8-bit data was encapsulated into a 10-bit frame. Each frame began with a Start Bit (Logic 0) and concluded with a Stop Bit (Logic 1). The plot visually confirmed these transitions between characters.

Conclusion:

The primary objective of this task was to differentiate between synchronous and asynchronous transmission at the physical and data link layers. In the Synchronous mode, the data is sent as a raw bitstream. While efficient in terms of overhead, this method requires the receiver and transmitter to share a common clock to prevent "bit-slipping." In contrast, the Asynchronous implementation utilized a 10-bit framing protocol. By adding a logic '0' at the start and a logic '1' at the end, we created a self-synchronizing signal. This allows the receiver to detect the beginning of a character even if the clocks are not perfectly aligned. The bit duration was strictly maintained at 1 second per bit. By using 100 samples per bit, the resulting digital waveform was smooth and clearly depicted the transitions between high (1V) and low (0V) states. During the simulation, it was noted that the Asynchronous method is less efficient than the Synchronous method. The total time on the x-axis for Asynchronous transmission is longer than the Synchronous one for the same message, because of those extra bits.

References:

- [1] MathWorks, *MATLAB Documentation*, Natick, MA, USA: MathWorks. [Online]. Available: <https://www.mathworks.com/help/>
- [2] Stormy Attaway, *MATLAB: A Practical Introduction to Programming and Problem Solving*, 6th ed. Boston, MA, USA: Butterworth-Heinemann, 2019